VIT
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

School of Computer Science and Engineering

Fall Semester 2026-27

BCSE412L Parallel Computing

SLOT: C1+TC1 **Faculty: Narayanan Prasanth**

Case Study Report: CS10 Parallel Machine Learning

Process-Level Parallelism for Machine-Learning Workloads in Python: K-means, k-NN and a HOG Preprocessing Pipeline on an Apple M1

Divyaraj (23BCE0722), Batch CS10

---

1. Abstract

In order to illustrate the process of executing three machine learning algorithms in parallel, we utilized 60000 images from the MNIST dataset using Lloyd K-means clustering algorithm, brute force k-nearest neighbor classification and HOG features preparation algorithms by applying Python multiprocessing with the shared memory technique on 8 core Apple M1. The calculations were performed in parallel version of the algorithms that included 1, 2, 4 and 8 worker processes and each configuration was compared with a sequential execution of the same algorithm. One 'warm-up' run and five timed runs were conducted for every configuration, which means 840 timed runs in total and 1854 correctness checks. At 8 worker processes, we obtained with K-means end-to-end speedup of 2.29, k-NN with partitioned training set achieved speedup of 3.29 and stage parallel preprocessing achieved speedup of 3.47. Unfortunately, the parallel efficiency never goes below 0.29 or above 0.43 and K-means executed on small 64 dimensional synthetic data has lower performance compared to sequential run. Additionally, during our experiments, we found out that our memory sampler was slowing down sequential runs by up to 11%, which resulted in situations when runs with single worker were executing faster than in an actual sequential execution.

2. Introduction and Literature Survey

The processing of tasks in machine learning, such as clustering, nearest neighbor search, and feature extraction is usually performed on several input samples at the same time, which makes these tasks suitable for parallel execution. It is notable that nowadays, even a typical laptop has six or more core processors, which remain unused by one Python process. It should be noted that increasing the number of processors does not increase the speed of parallel execution proportionately. Amdahl [1] demonstrated that the serial component in programme execution hinders obtaining the maximum speedup irrespective of the number of processors in use, whereas Gustafson [2] claimed that the increase in the problem size along with the number of processors results in the parallel components prevailing and thus, enhancing speedup.

We utilized the Lloyd algorithm [3] for K-means, which assigns points to the nearest centroid during every iteration, following which the centroids are recalculated. The assignment step takes the bulk of the processing time. Srivastava and Nawfal [7] have studied parallelizing this algorithm for big data clustering, applying the same concept to parallelize the assignment step across cores as we have done in our S01. In the case of k-NN, the main issue is whether to split query or training data; PANDA [8] divides data among the nodes of a distributed cluster and merges local nearest neighbor results, while Faiss [9] is capable of performing brute-force and approximate searches on GPUs at the scale of billions, while we are comparing both splits on a single CPU in S02.

In terms of preprocessing, it has been shown recently that the input pipeline is often the actual bottleneck in machine learning training. Mohan et al. [10] have proven that much of the training time is taken waiting for

the data. Plumber [11] is a tool made for finding and fixing bottlenecks in input pipelines, Isenko et al. [12] have shown that the place and methods of preprocessing lead to some unexpected trade-offs between throughput, storage, and costs of running processes, Ray Data [13] is running preprocessing as a streaming pipeline on several different machines. In our case, in S03 pipeline we make use of HOG features [4] on MNIST [5], and the pipeline is built using NumPy [6], scikit-learn [14] and scikit-image [15].

The majority of the studies focus on either GPUs or large clusters of machines, and only few of them look at the performance of simple process-level parallelism from the Python programming language on a single laptop with CPUs with varying performance. This case study presents the results of testing three types of workloads with this method, where we separated the computational and overhead parts, compared our findings with those of sequential applications, and controlled the measurement process to identify any potential errors.

3. Problem and Research Question

We have a task to reduce the required time for three distinct workloads (K-means, brute-force k-NN and HOG) by the utilization of multiple Python processes and execution on an Apple M1 machine that has a total of eight cores (four performance cores and four efficiency cores). To execute and get the results of the algorithms, we will need to obtain three different configurations of input data (MNIST subsets including images 10K, 30K and 60K, along with 64-dimensional blobs for K-means, and test queries for k-NN (including 1,000, 5,000 and 10,000 queries)).

The research question is about the performance improvement obtained from process-level parallelism with three workloads running on a laptop CPU based on heterogeneous architecture. In S01, we will investigate the scalability of the data-parallel assignment step. For S02, we will compare the time required for query-based or training-set splitting. Finally, in S03, we will compare the performance of a stage-parallel system and a process pipeline.

Before running the experiments we stated these hypotheses:

- **H1:** workloads with more computation per synchronization point will scale better, so K-means, which has a barrier in every iteration, will scale worst and HOG preprocessing will scale best.
- **H2:** speedup will improve a lot up to 4 workers and only a little from 4 to 8 workers, because only 4 of the 8 cores are performance cores.
- **H3:** query-partitioned k-NN will be faster than train-partitioned k-NN because it does not need a merge step in the parent.
- **H4:** small problems will gain little or nothing from parallelism because the process start-up cost will be close to the total work.

4. Methodology

4.1 Experimental environment

All experiments were carried out on one laptop, which are shown in Table 1. The Apple M1 is a heterogeneous chip: it has 4 cores that are high performance (P) and 4 cores that are energy-saving (E), while all cores share 8 GB of unified memory, so with 8 workers some of them are necessarily placed on the slower cores. No GPU was used, and energy was not measured in this work.

Table 1. *Hardware and software environment (from results/env.txt).*

Component	Configuration
Machine / CPU	MacBook Air, Apple M1: 8 cores = 4 performance + 4 efficiency
Memory	8 GiB unified memory
Operating system	macOS 26.6.2 (Darwin 25.6.0, arm64)
Python	Version 3.14.2; usage of `multiprocessing` with spawn as start mode (default on macOS)
Libraries	NumPy 2.5.4 (BLAS: Apple Accelerate), scikit-learn 1.9.1, scikit-image 0.26.0, psutil 7.2.2, pandas 3.0.6, matplotlib 3.11.2
BLAS threads	1 thread set per process (OMP_, VECLIB_MAXIMUM_, OPENBLAS_, MKL_NUM_THREADS set to 1)
Power state	AC power; Low Power Mode off
GPU / energy	Not used / not measured

4.2 Baselines

The speedup values reported here are compared to a sequential implementation of the particular algorithm in solely one process and without utilizing a process pool, so the reference time for each speedup measure is the average of the measurements obtained during 5 separate runs. It is necessary to mention that NumPy uses a multithreaded BLAS library to perform matrix operations so that a number of the `OMP_NUM_THREADS`, `VECLIB_MAXIMUM_THREADS`, `OPENBLAS_NUM_THREADS`, and `MKL_NUM_THREADS` variables are set equal to one in the entry scripts before NumPy is imported.

The second one, a diagnostic baseline sequential-chunked one, performs the same task decomposition on a single process undertaken by eight workers (same chunk boundaries, same task kernel, same order of reduction). This is never used to compute speedup. It is only used to check the impact of chunking on performance; for example, by testing whether cache locality affects performance.

4.3 Parallel strategies

All of the parallel implementations leverage the multiprocessing functionality available in Python and run with either 1, 2, 4 or 8 worker processes. Input arrays are transferred to the fixed-sized memory segments of `multiprocessing.shared_memory` in the beginning of every run, which are then attached to by the worker processes during the initialization of the pool. Therefore, there's no need to pass any serialized data, with the exception of the bounds of assignments (and current centroids for the K-means). Squared Euclidean distance is calculated using the following formula: $\|a\|^2 + \|b\|^2 - 2a \cdot b$, such that the two operations with the most considerable computational cost in S01 and S02 consist in matrix multiplication.

S01: K-means (also known as Lloyd's algorithm [3]). The data is separated into fixed blocks with 250 rows, and each worker receives an uninterrupted set of blocks. During the iterations the parent process forwards the current centroids to the workers and the workers assign their rows to the closest centroid, write the labels into shared memory, and return the partial sums, counts, and inertia per cluster; the parent process then reduces the partial results and computes new centroids (see Figure 1). This reduction is a global barrier that occurs once per iteration, with the same pool used in all iterations. The initial centroids are k rows chosen with an identical seed value (0) which is also the same for all alternatives. The iteration stops once the squared shift of centroid becomes less than 10^{-4} times the average variance of features or the iteration count reaches the limit of 50.

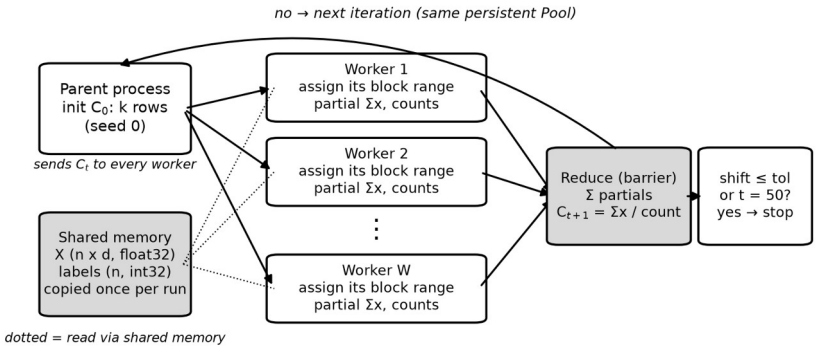


Figure 1. *S01 parallel K-means: data-parallel assignment stage with a persistent pool of worker processes and a reduction barrier in the parent in every iteration.*

S02: k-NN. Implementing a basic k-NN algorithm with k = 5, we can calculate distances of one query block containing 125 queries against one training chunk of 1,250 rows, using a single matrix computation. For the sake of this application, we used two different types of decomposition (illustrated in Figure 2). In the query-partitioned approach, the query blocks are assigned to the separate workers first. Once the worker receives its blocks, it will retrieve predicted labels for every query by matching it against the entire training set. The train-partitioned approach operates in a slightly different manner. In this method, the training set is split among several workers. Each of them will compute distances between all queries and its own share of the training set, returning for every query not the predicted labels, but only five nearest candidates (distance and global index), which are to be merged by the parent at a later stage. In this case, the order of candidates is determined based on the pair consisting of (distance, training index).

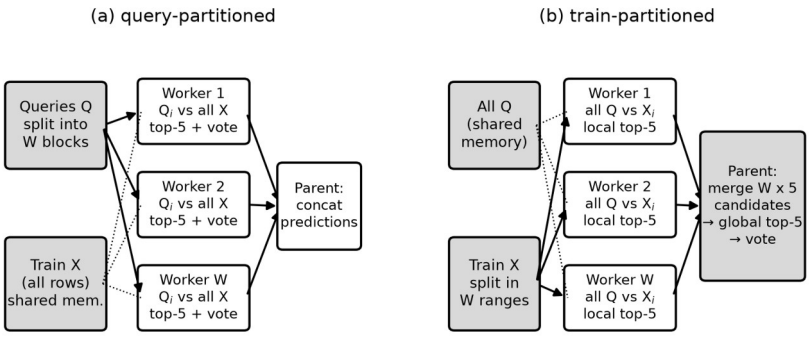


Figure 2. *Parallel k-NN in S02: (a) shows a method in which queries are divided among the worker processes that actually do the work, and (b) shows a method in which the training set gets divided and, after processing, the local top-5 results are merged in the parent process.*

S03: Preprocessing pipeline. The preprocessing pipeline for MNIST images includes four stages: (1) reading uint8 images from 1,000-image .npz shards stored on the disk, (2) normalization to [0, 1], (3) standardizing each image so that it has zero mean and unit variance, and (4) calculating the Histogram of Oriented Gradients (HOG) features [4] using scikit-image [15] (which employs 9 orientations, cells of size 7 × 7 pixels, 2 × 2-cell blocks, which results in 324 features for each image). In the stage-parallel approach, each stage works in parallel and has barriers (see Figure 3a). In the pipeline approach, each stage operates in its own process, and there are bounded queues transporting batches of 1,000 images; only the HOG stage is replicated W times, so the pipeline will have W + 3 processes (see Figure 3b). The standardization is carried out per image, not on a feature level so that a batch can be processed independently as required by the pipeline.

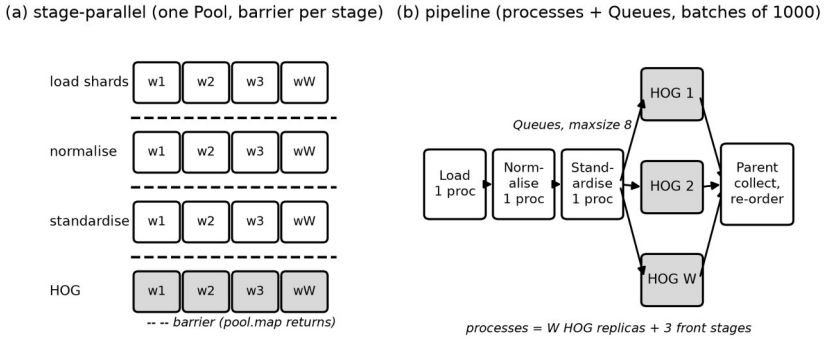


Figure 3. *The different S03 preprocessing methods: (a) stage-parallel along with the occurrence of a barrier after each stage; and (b) process pipeline with W HOG replicas.*

4.4 Data and specifications

The MNIST dataset [5] which consists of 70,000 images of handwritten digits each of size of 28 × 28 pixels was downloaded only once from OpenML and is stored locally. The time used for downloading is not included in the measurements. The values of pixels are kept in float32 format within the range of 0 to 1. The parameters of all the configurations are shared in Table 2. Overall, the number of configurations was 168 with 840 timed runs completed (S01: 240, S02: 450, S03: 150).

Table 2. *Datasets, parameters and variants per sub-problem (w = 1, 2, 4, 8 for every parallel variant).*

	Data and sizes	**Parameters**	**Variants**
S01	MNIST (784 features) and `make_blobs` Gaussian blobs (64 features, 10 centres, seed 0); 10k, 30k and 60k samples each	k = 10, plus k = 5 and 20 on 30k MNIST; at most 50 iterations; tolerance 10^{-4}; 250-row blocks	sequential, sequential-chunked, multiprocessing
S02	Training: 10k, 30k and 60k MNIST images. Queries: first 1,000, 5,000 and 10,000 test images	k = 5, Euclidean distance; 125-query blocks; 1,250-row training chunks	sequential, sequential-chunked, query- and train-partitioned
S03	First 10k, 30k and 60k MNIST training images in 1,000-image shards	HOG with 9 orientations, 7 × 7 cells and 2 × 2 blocks (324 features); batches of 1,000; queue capacity 8	sequential, sequential-chunked, stage-parallel, pipeline

4.5 Measurement protocol and metrics

Every configuration was executed once as a warm-up, which was checked for correctness and then discarded, followed by five timed runs (Figure 4). Reported values are means; error bars and ± values are standard deviations over the five runs. Timing used `time.perf_counter` within a process and `time.time` for timestamps compared across processes. Execution_Time is the wall-clock time of the whole algorithm, including shared-memory setup and process-pool start-up and shutdown, and is decomposed as follows:

- **Scheduling:** creating and shutting down the pool (or pipeline processes) plus the dispatch latency from submitting a parallel map to the first task starting.
- **Communication:** copying data into and out of shared memory, plus the delay between the last task finishing and the parent receiving the results; for the pipeline, the explicit serialisation and deserialisation of every batch.
- **Synchronization:** for each parallel map, the span from the first task start to the last task end minus the busiest worker's compute time, i.e. idle time at the barrier; for the pipeline, the longest time a HOG process waited for input.
- **Computation:** the remainder of the execution time.

Derived metrics are computed from the raw times by `analysis/analyze.py`, not entered manually: speedup $S = T_{seq} / T_{par}$, efficiency $E = S / W$, and compute-only speedup $S_c = T_{seq} / (T_{par} - T_{sched})$, which excludes process start-up and dispatch and therefore isolates how well the computation itself scales. Peak memory is the resident set size of the parent and its worker processes, sampled every 20 ms. Before the final sweep other applications were closed, Low Power Mode was confirmed off with the laptop on AC power, and the sweep started only after Spotlight indexing had stopped (Section 6). The complete sweep took 22.5 minutes (console output in Appendix, Figure B8).

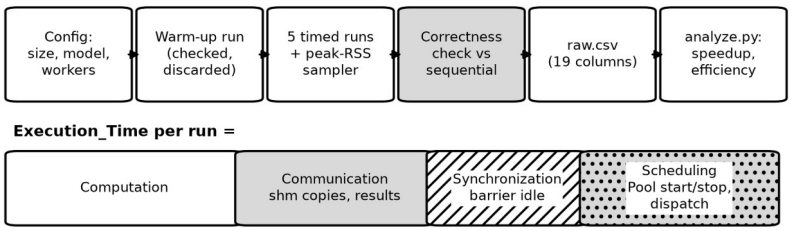


Figure 4. *Measurement protocol per configuration (top) and the decomposition of each run's execution time (bottom).*

4.6 Correctness validation

Every run, whether it is warm-up or chunk-oriented run, is checked against the sequential result of the configuration: K-means must give identical clusters and number of iterations; k-NN must produce identical classifications, which facilitates identical accuracy; preprocessing must result in identical HOG feature matrices. If a check fails, the sweep is aborted. Our approach guarantees perfect matching: distances are calculated in the same fixed, globally aligned blocks, so all methods carry out byte identical matrix multiplications, and partial sums are calculated in double precision. All of the 1854 sets of checks were successful (Appendix, Figure B9).

5. Results and Discussion

All speedups below are relative to the pure sequential baseline at the same problem size (Section 4.2); w denotes the number of worker processes. Each figure's error bars are standard deviations over five runs.

5.1 Combined results

Table 3 summarises the largest problem size (60,000 samples; 10,000 queries for k-NN) for all three sub-problems, and Figure 5 contrasts end-to-end with compute-only speedup.

Table 3. *Combined results at 60,000 samples (k-NN: 10,000 queries). S = end-to-end speedup vs pure sequential; S_c = compute-only speedup; E = efficiency; overhead = (communication + synchronization + scheduling) / execution time. Means of 5 runs.*

Sub-problem / variant	T_{seq} (s)	S w=1	S w=2	S w=4	S w=8	S_c w=8	E w=8	Overhead w=8
S01 K-means, MNIST, k=10	2.662	0.95	1.63	2.22	2.29	2.44	0.29	10%
S01 K-means, Blobs-64d, k=10	0.160	0.62	0.80	0.88	0.66	1.92	0.08	71%

Sub-problem / variant	T_{seq} (s)	S w=1	S w=2	S w=4	S w=8	S_c w=8	E w=8	Overhead w=8
S02 k-NN, query-partitioned	6.884	0.98	1.71	2.55	3.06	3.32	0.38	10%
S02 k-NN, train-partitioned	6.884	0.98	1.73	2.58	3.29	3.58	0.41	11%
S03 preprocessing, stage-parallel	5.261	0.96	1.83	2.99	3.47	3.94	0.43	15%
S03 preprocessing, pipeline	5.261	0.98	1.83	2.84	3.28	3.37	0.41	18%

The highest end-to-end speedup was 3.47× (S03 stage-parallel, w = 8), followed by 3.29× for train-partitioned k-NN and 2.29× for K-means on MNIST. No configuration reached 50% efficiency at eight workers (maximum 0.43). K-means on the 64-dimensional synthetic data never ran faster than sequential (best 0.88× at w = 4). At w = 1 every parallel variant was slower than sequential (0.25×–0.98× across all 32 w = 1 configurations), which bounds the fixed cost of using a process pool.

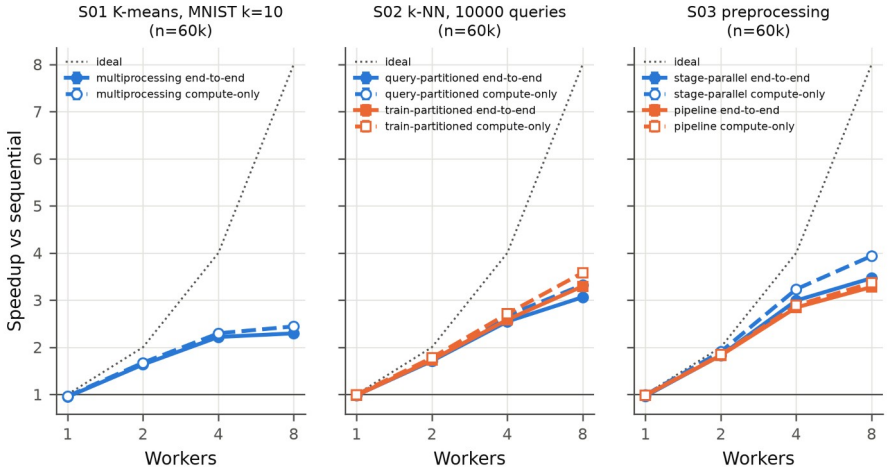


Figure 5. *End-to-end (solid) vs compute-only (dashed) speedup at the largest problem size for each sub-problem.*

5.2 S01: Parallel K-means

For k = 10 in MNIST, the speedup increased with the size of the problem: for w = 4, there were 1.33 times for 10k, 1.88 for 30k and 2.22 for 60k. Increasing the number of workers from four to eight resulted in a drop in speedup for 10k (1.33 → 1.09) and 30k (1.88 → 1.82), while the impact at 60k was marginal (2.22 → 2.29) (Figure 6). The compute-only speedup achieved 2.30 (w = 4 at 60k), while at w = 8 it raised to 2.44, which means that pool start-up at this scale contributes little: the scheduling process consumed only 0.070 s out of total time of 1.161 s at w = 8, and all overhead components accounted for 10 percent of this figure. At 30k, the number of clusters has little significance as the maximum speedups recorded were 1.77, 1.88 and 1.86 correspondingly for k equals 5, 10, and 20, all for w = 4.

Running on the synthetic data set of 64 dimensions, the cumulative sequential execution was mere 0.160 s for 60k (17 iterations). When w = 8, scheduling process accounted for 0.160 of total of 0.244 s parallel run; maximum end-to-end speed up obtained was 0.88 (when w = 4), while compute-only speed up was 2.22 (when w = 4). The number of iterations was the same for every variant and worker count: blobs had 22, 23 and 17 iterations for 10k, 30k and 60k; MNIST (k = 10) had 46, 43 and 50 iterations at the same sizes; and at 30k MNIST had 33 (k = 5) and 50 (k = 20) iterations. Two MNIST configurations (60k, k = 10 and 30k, k = 20) terminated after performing 50 iterations instead of reaching tolerance level used (Appendix, Figure B2).

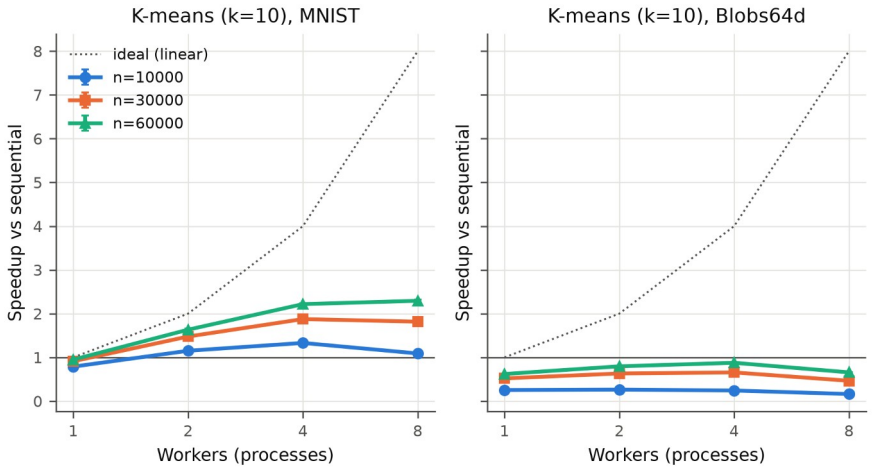


Figure 6. *S01 K-means speedup vs workers for MNIST (left) and 64-d blobs (right), k = 10.*

K-means stabilizes at 4 workers due to the organization of the work in each iteration. The parent needs to send out the centroids, wait for all workers to finish, and then perform the reduction itself. During this process, in up to 50 iterations, there is the possibility of 50 such rounds of work for each run. The serial part and the wait at the barrier act as the serial part of Amdahl's law [1], meaning that even with more workers the only thing that improves is the previously parallel work. Moreover, workers 5-8 run on the less productive efficiency cores so the barrier has to wait for the least efficient worker in every iteration.

Larger datasets scale better because the extraneous costs stay relatively constant as the complexity grows according to the number of data points, as explained by Gustafson [2]. The blobs data set is not successful because it has only 64 instead of 784 features and has no more than 17 iterations, so the entire sequential process takes 0.160 s. This is the same as the time it takes for the scheduling in the case of 8 workers, and once start-up is removed the compute-only speedup is 2.22×, so the computation itself does scale. Similar to Srivastava and Nawfal [7], we have come across the fact that the step of assignment parallelises well but the fixed costs for starting the processes decide whether it is worthwhile.

5.3 S02: Parallel k-NN

Table 4 presents a comparison of two strategies using w=8 for all sizes of the training sets and numbers of queries. Using 10,000 queries and a training set of 60,000 images, the speedup factor in the case of the query-partitioned k-NN was 3.06 and for the train-partitioned k-NN it was 3.29 (Figure 7); the speedup factor for the case with computations only is 3.32 for the query-partitioned k-NN and 3.58 for the train-partitioned one. The throughput went from 1,453 queries/sec for the sequential execution to 4,785 queries/sec (the case with train-partitioned k-NN at w=8) (Appendix, Figure B5). The train-partitioned k-NN was better at 60k training images for all query counts, while the query-partitioned one was more effective at 10k training images for all queries (the difference being only 0.01 at 1000 queries).

Table 4. *k-NN strategies at w = 8: end-to-end speedup S, compute-only speedup S_c and peak resident memory (query-partitioned / train-partitioned).*

Train size	Queries	T_{seq} (s)	S query	S train	S_c query	S_c train	Peak RSS (MB)
10,000	1,000	0.122	0.52	0.51	1.86	1.80	1,461 / 1,229
30,000	1,000	0.334	0.99	1.05	2.04	2.18	2,115 / 1,192
60,000	1,000	0.712	1.44	1.72	2.22	2.90	3,168 / 1,276
10,000	5,000	0.608	1.67	1.56	3.16	2.75	1,375 / 981
30,000	5,000	1.660	2.38	1.88	3.14	2.36	1,967 / 1,086
60,000	5,000	3.540	2.72	3.04	3.14	3.54	3,106 / 1,234
10,000	10,000	1.208	2.35	2.08	3.44	2.93	1,264 / 1,171
30,000	10,000	3.362	2.90	2.71	3.41	3.14	1,825 / 1,109
60,000	10,000	6.884	3.06	3.29	3.32	3.58	3,112 / 1,407

With only 1,000 queries the computation is small relative to start-up: at 10k training images neither strategy beat sequential (best 0.68×), and eight workers were slower than four in most 1,000-query configurations. Scheduling cost was nearly constant at 0.16–0.18 s per run at w = 8, regardless of problem size. Memory use

differed between the strategies: at 60k training images, 10,000 queries and w = 8, peak resident memory was 3,112 MB for query-partitioned versus 1,407 MB for train-partitioned. Classification accuracy was identical for all strategies and worker counts, ranging from 91.6% (10k training images, 1,000 queries) to 96.9% (60k, 10,000 queries).

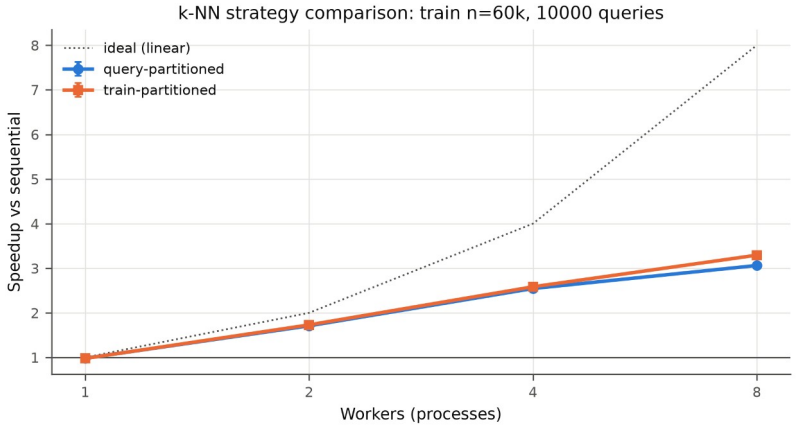


Figure 7. *S02 strategy comparison at 60k training images and 10,000 queries.*

In the train-partitioned approach, each worker merely processes its own fraction of the training dataset. This way, with a total of 60,000 training images, each worker works with 7,500 rows only instead of processing all 60,000 images which proves to be much more feasible in terms of cache usage and memory bandwidth. The merging of candidates at 8 × 5 per query in the parent is, therefore, negligible in comparison with the distance computation process. Using 10,000 and 30,000 of the training images the shares are so small that the query-partitioned method performs effectively, and there the additional merging of the candidates for each query puts the train-partitioned method mostly slightly behind.

The difference in memory arises from how much of the shared training set is accessed by each worker; in the case of query-partitioned approach where every worker needs to access the whole 188 MB training dataset from a shared memory, there is repeated counting of those shared volumes in each process; however, in the case of the train-partitioned strategy, each worker only accesses its share of the data. Therefore, the peak results are about 1.7 GB lower despite no additional copies of the dataset being created. As for the performance of the 1,000-query runs, the gain is almost negligible because a sequential performance took only 0.12 to 0.71 seconds, while the scheduling time of execution itself is about 0.17 seconds if we have 8 workers executing the tasks. PANDA [8] has a similar approach since it uses the partitioning of data across nodes and local result merging just like our train-partitioned strategy, while Faiss [9] employs brute-force distance calculations using matrix multiplication, hence the same design choices appear at much bigger scale.

5.4 S03: Parallel preprocessing pipeline

HOG extraction dominated the sequential pipeline at every size, taking 98.2%, 97.8% and 97.3% of the time at 10k, 30k and 60k images; loading, normalisation and standardisation together took under 3% (Figure 8). At 60k images the stage-parallel variant reached 3.47× and the pipeline 3.28× at w = 8, i.e. 39,518 and 37,371 images/s versus 11,404 images/s sequentially (Appendix, Figure B7). Unlike S01 and S02, eight workers were faster than four for both variants at 30k and 60k; at 10k, w = 8 was again slightly slower than w = 4 (stage-parallel 2.08× vs 2.19×; pipeline 1.73× vs 1.80×).

Table 5. *S03 per-stage times at 60k images (s). Stage-parallel: wall time per stage; pipeline: busy CPU time summed over the stage's processes (w = 1 / w = 8).*

Stage	Sequential	SP w=1	SP w=2	SP w=4	SP w=8	Stage S w=8	Pipeline busy w=1 / w=8
load	0.022	0.131	0.106	0.108	0.146	0.15×	0.034 / 0.073
normalise	0.020	0.032	0.018	0.011	0.017	1.21×	0.023 / 0.059
standardise	0.099	0.080	0.049	0.042	0.041	2.39×	0.063 / 0.157
HOG	5.120	5.161	2.646	1.535	1.223	4.19×	5.140 / 9.967

Table 5 breaks the 60k run down by stage. In the stage-parallel variant the HOG stage scaled to 4.19× at w = 8, standardisation to 2.39× and normalisation to 1.21×, while loading became 6.6 times slower than sequential (0.146 s vs 0.022 s). In the pipeline, the summed busy CPU time of the HOG replicas rose from 5.14 s with

one replica to 9.97 s with eight (1.94×) although the work was identical. The overhead profiles also differ: at w = 8 the stage-parallel run spent 0.181 s on scheduling and 0.026 s on communication, whereas the pipeline spent 0.042 s on scheduling, 0.152 s on communication (batch serialisation) and 0.089 s waiting for input.

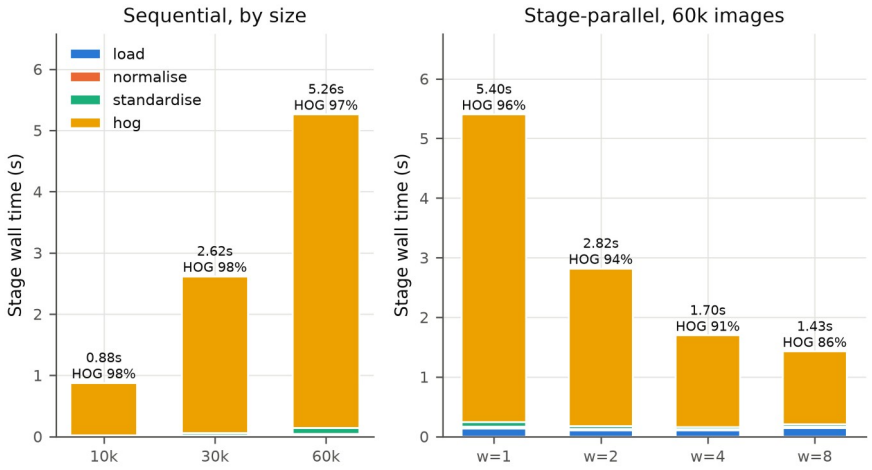


Figure 8. *S03 stage contribution: sequential by size (left) and stage-parallel at 60k images by worker count (right).*

To summarize the efficiency of different kernels tested in this paper, it is observed that HOG scales much better than all the other kernels since all the images are processed independently from each other and thus obtaining a better overall performance. Since HOG uses more than 97% of the overall time, the whole pipeline will scale according to the HOG performance as well. Loading gets slower when parallelised because dispatching the tasks and attaching shared memory for the small shards takes more time than reading them, but Amdahl's law [1] shows that the influence on overall efficiency will be negligible since the loading takes less than 3% of the total time.

The pipeline is somewhat behind stage-parallel since every batch is required to be serialized and moved through the queue, and with 8 workers, it uses up to 11 processes on the 8 cores. Moreover, the pipeline requires time both to fill and to drain. The CPU time for HOG with 8 copies is 1.94 times longer than with 1 copy, which suggests that some copies are operating on less efficient cores while other cores have to share the main resources, but we did not verify their placement. This is in line with the studies devoted to data stalls and pipelines [10]–[13].

5.5 Overheads across the three sub-problems

The breakdown of the average execution time for the largest size is represented in Figure 9. The overhead proportion (i.e., communication plus synchronization plus scheduling, expressed as a part of execution time) varied from 10% (K-means used with MNIST dataset with the size of 60k; k-NN with query partitioning for the size of 60k with 10,000 queries to execute) to 86% (K-means applied for blobs dataset with the size of 10k) for all settings at the value of w = 8. Scheduling was the main component of overhead for all pool-based variants, and for all trials the scheduling time ranged from 0.07 s to 0.18 s (0.16-0.18 s for k-NN and stage-parallel preprocessing regardless of size) for w = 8.

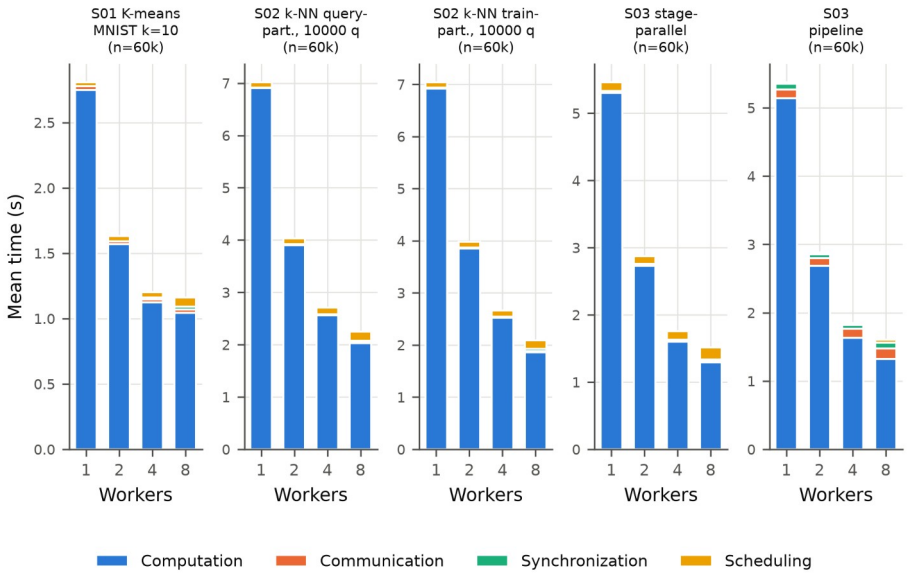


Figure 9. *Execution-time decomposition (mean of 5 runs) at the largest problem size for each sub-problem and variant.*

5.6 Investigation: parallel runs with one worker faster than sequential

An earlier sweep, which used three timed runs and the original memory sampler, showed several w = 1 configurations running 6–10% faster than the sequential baseline. This should not happen, because the work is identical and the parallel version only adds overhead. Two explanations were tested.

Chunking / cache locality. The sequential-chunked baseline (Section 4.2) changed run time by between −1.6% and +1.0% relative to pure sequential across all 20 comparisons, i.e. within noise, so chunking does not explain the effect.

Interference in Measurement. The original memory sampler was a thread that had to be run in the parent process and check all approximately 500 running processes every 20 ms (3.2 ms/sample), while occupying the Python global interpreter lock. Only in-process (sequential) runs were competing against this thread; pool workers are separate processes. Table 6 shows the sequential run times at 60k with the memory sampler in each mode, measured in interleaved order. As seen in the table, the original memory sampler (scan) slowed the sequential runs by 11.0% (K-means), 4.5% (k-NN), and 10.0% (preprocessing), whereas with the replacement (track) the difference from running without a sampler was less than 1%. In the end configuration using the replacement, no configuration with w = 1 showed faster performance than running sequentially (0.25-0.98 times as quickly). All the values, which were mentioned in this report, are taken from the final configuration of measurements since an earlier one had shown misleading results as well.

Table 6. *Pure sequential run time at 60k (s, mean of 3 interleaved runs) with the memory sampler off, in the corrected mode (track) and in the original mode (scan). Source: results/sampler_diagnostic.csv (console output in Appendix, Figure B10).*

Workload	off	track	scan (original)	track vs off	scan vs off
S01 K-means (MNIST, k=10)	2.661	2.666	2.954	0.2%	+11.0%
S02 k-NN (1,000 queries)	0.701	0.707	0.732	0.9%	+4.5%
S03 preprocessing	5.209	5.196	5.729	-0.3%	+10.0%

5.7 Comparative discussion

The three workloads at 60k samples have the following speedup ranks among each other at 8 workers, namely S03 (3.47×) > S02 (3.29×) > S01 (2.29×), which is in accordance with the total number of computations performed per synchronization. K-means runs between 1 and 50 synchronization points per run, and k-NN executes one parallel mapping process per run, while preprocessing takes place in four steps, of which HOG is the most time-consuming and fully independent. The efficiency rate for stage-parallel preprocessing was 0.75 (4 workers), that for k-NN being 0.65 (4 workers), while the efficiency rate for K-means was only 0.55 (4 workers).

H1 has been validated as K-means performed the worst among the three and HOG was the best. **H2** is partially validated as K-means performed just 3% better with the change of the worker count from 4 to 8 and it was even slower with the higher worker count at smaller sizes, while k-NN and preprocessing gave good

improvements of 15% to 27% with the efficiency cores. **H3** fails with larger data cases as train-partitioned k-NN was faster for all query counts at 60k training images (3.29 times vs 3.06 times with 10k queries) and consumed less than half the memory used by the query-partitioned k-NN. **H4** also proves to be valid since the blobs dataset could not surpass sequential processing in any mentioned case (the best being 0.88 times).

Computational-only speedup (not taking into account start-up) shows when start-up affects the end result: the compute-only speedup for the blobs was 2.22x, but the end-to-end speedup remains less than 1, while on the MNIST at 60k the numbers were close (2.29x and 2.44x for K-means), implying process start-up makes a difference only when actual work takes literally a fraction of a second. The single-worker study led us to believe that measuring tools could introduce errors into the outcome where a sampling thread spending no more than a couple of milliseconds in every 20 ms led to a baseline slower by 11% and a false speedup; hence reference measurements have to be done with these same tools enabled and checked for correctness. The speed-ups we report, compared to the literature [7]-[13], were lower in absolute terms because we used one laptop and Python processes instead of clusters, GPUs or compiled code; the bottlenecks are, however, the same: synchronization in K-means, the way data is partitioned in k-NN, and data movement between stages of preprocessing.

6. Limitations and Threats to Validity

- **Implementation with a single heterogeneous machine.** The computer used was an 8-core Apple M1 containing 4 performance cores and 4 efficiency cores. Some tasks are run on slower cores when eight workers are employed, but core allocation is taken care of by the macOS scheduler. The findings may not be valid for other types of machines as the CPU being used differs in this case.
- **Python parallelism implementation relies on processes alone.** The design already includes spawn, serialization, and shared memory setup; optimized thread-based implementations would be expected to have other overheads. The role of any GPUs and power consumption are disregarded here.
- **BLAS library.** The matrix multiplications are carried out in Apple Accelerate; each process relies on one thread while internal performance could not be established.
- **Data and parameters.** The dataset is MNIST plus one synthetic dataset made of no more than 60,000 samples. In two of the K-means configurations, the algorithm stopped at the 50-iteration limit instead of converging.
- **Measurement noise.** For each configuration, 5 runs were conducted which led to a coefficient of variation with a median of 0.6% and a maximum of 16.9% (train-partitioned k-NN with 30K training images – 1,000 queries, w = 4).
- **Measurement interference – found and treated.** The original memory sampler led to a bias of 4.5–11% in the sequential baseline (Section 5.6). It was replaced, and its influence was evaluated before the final testing. Memory_Use includes the version of MNIST copied from the parent (around 400 MB).
- **Background activity.** Only Finder and terminal were running – pre-flight test registered through results/env.txt. Low Power Mode was off and laptop was connected to the electric current. During the first test, 13% of the CPU was used by the Spotlight processing, so the sweep started only after the second test which showed the CPU 97.3% idle (load average = 1.46). Colima (Docker) machine was functioning during all tests with 0% of CPU used and about 340 MB of resident memory.
- **Timing granularity and definitions.** The decomposition results are based on using timestamps which are compared among processes so anything shorter than a millisecond is estimated. Synchronization for the pool variants means barrier idle time but for the pipeline it means input waiting time, which means that these two values are not comparable.
- **CPU utilization is not provided.** CPU utilization was not tracked separately for S03. For instance, on the M1 chip, efficiency cores do not perform nearly as much work as performance cores when they are busy and macOS does not provide any mechanisms by which user processes can occupy a specific core or see which core they operate on, so a simple utilization number would not indicate the real productive work done. Instead, we provide statistics on efficiency and synchronization time showing how much time was used productively.

7. Conclusion and Future Work

To summarize the answer to the research question, it appears that process-level parallelism in Python provided benefits but not to their maximum potential. When using the M1 processor with 8 cores, we were able to

achieve speedup factors of 2.29 for K-means, 3.29 for k-NN and 3.47 for preprocessing of the data with 60k samples and 8 parallel processes on a single machine, which yielded decent but low efficiency of at most 0.43. The limitations were mainly the synchronization at each iteration, almost constant startup costs of between 0.07 and 0.18 s per run and the fact that efficiency-core takes longer to process the job than the other cores. In real life, it has been observed that on a laptop CPU this kind of parallelism is useful only when each execution of the single process takes at least several seconds and when the job requires little to no synchronization, while jobs that last less than approximately half a second lose their whole advantage due to startup costs. All outputs of the parallel processes match the sequential processing results from all runs.

In the future this work can be extended in these directions:

- Use threads in free-threaded Python 3.14 instead of processes, which would remove most of the start-up and data-copying cost.
- Restrict the workers to the 4 performance cores and compare this with using all 8 cores to measure the effect of the efficiency cores directly.
- Run the same workloads on the M1 GPU through Metal/MPS and measure the energy use with powermetrics.
- Keep one process pool alive across runs so that the start-up cost is paid only once, and try larger datasets with frameworks like Ray Data [13].
- Use approximate nearest neighbour search [9] and mini-batch K-means to reduce the work itself instead of only splitting it.

8. References

[1] G. M. Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," in *Proc. AFIPS '67 (Spring) Joint Computer Conf.*, Apr. 1967, pp. 483–485, doi: 10.1145/1465482.1465560.

[2] J. L. Gustafson, "Reevaluating Amdahl's law," *Communications of the ACM*, vol. 31, no. 5, pp. 532–533, May 1988, doi: 10.1145/42411.42415.

[3] S. Lloyd, "Least squares quantization in PCM," *IEEE Transactions on Information Theory*, vol. 28, no. 2, pp. 129–137, Mar. 1982, doi: 10.1109/TIT.1982.1056489.

[4] N. Dalal and B. Triggs, "Histograms of oriented gradients for human detection," in *Proc. 2005 IEEE Computer Society Conf. Computer Vision and Pattern Recognition (CVPR'05)*, 2005, vol. 1, pp. 886–893, doi: 10.1109/CVPR.2005.177.

[5] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998, doi: 10.1109/5.726791.

[6] C. R. Harris *et al.*, "Array programming with NumPy," *Nature*, vol. 585, no. 7825, pp. 357–362, Sep. 2020, doi: 10.1038/s41586-020-2649-2.

[7] A. Srivastava and M. Nawfal, "Parallelization of the K-means algorithm with applications to big data clustering," arXiv preprint arXiv:2405.12052, 2024, doi: 10.48550/arXiv.2405.12052.

[8] Md. M. A. Patwary *et al.*, "PANDA: Extreme scale parallel k-nearest neighbor on distributed architectures," in *Proc. 2016 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, May 2016, pp. 494–503, doi: 10.1109/IPDPS.2016.57.

[9] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, Jul. 2021, doi: 10.1109/TBDATA.2019.2921572.

[10] J. Mohan, A. Phanishayee, A. Raniwala, and V. Chidambaram, "Analyzing and mitigating data stalls in DNN training," *Proceedings of the VLDB Endowment*, vol. 14, no. 5, pp. 771–784, Jan. 2021, doi: 10.14778/3446095.3446100.

[11] M. Kuchnik, A. Klimovic, J. Simsa, V. Smith, and G. Amvrosiadis, "Plumber: Diagnosing and removing performance bottlenecks in machine learning data pipelines," in *Proceedings of Machine Learning and Systems (MLSys)*, vol. 4, 2022, pp. 33–51.

[12] A. Isenko, R. Mayer, J. Jedele, and H.-A. Jacobsen, "Where is my training bottleneck? Hidden trade-offs in deep learning preprocessing pipelines," in *Proc. 2022 International Conference on Management of Data (SIGMOD '22)*, Jun. 2022, pp. 1825–1839, doi: 10.1145/3514221.3517848.

[13] F. S. Luan *et al.*, "Ray Data: A dynamic and scalable system for heterogeneous data processing," arXiv preprint arXiv:2501.12407, 2025 (to appear in USENIX NSDI '27), doi: 10.48550/arXiv.2501.12407.

[14] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, no. 85, pp. 2825–2830, 2011.

[15] S. van der Walt *et al.*, "scikit-image: Image processing in Python," *PeerJ*, vol. 2, p. e453, Jun. 2014, doi: 10.7717/peerj.453.

Appendix A. Reproducibility

Source code, raw data and plots are available at https://github.com/divyaraj24/PC_CS10_23BCE0722 (README.md lists every file). The complete experiment is reproduced with:

```
python3 -m venv .venv && .venv/bin/pip install -r requirements.txt
.venv/bin/python -m common.data                    # download + cache MNIST (one-off)
.venv/bin/python run_all.py --smoke                # 1k-sample smoke test (~30 s)
.venv/bin/python analysis/sampler_diagnostic.py # optional: sampler cost (~2 min)
.venv/bin/python run_all.py                        # full sweep (~22.5 min) -> results/raw.csv
.venv/bin/python analysis/analyze.py               # derived.csv, summary_table.csv/.md
.venv/bin/python analysis/plots.py                 # plots/*.png + *.svg
.venv/bin/python analysis/findings.py              # results/findings.md
.venv/bin/python report/make_figures.py            # diagrams + console figures
```

Raw measurements (results/raw.csv) contain one row per timed run with the 19 columns required by the case-study template; speedup and efficiency are left blank there and computed by analyze.py into results/derived.csv. Plots are provided as editable SVG together with the script that generates them.

Appendix B. Supplementary figures and console output

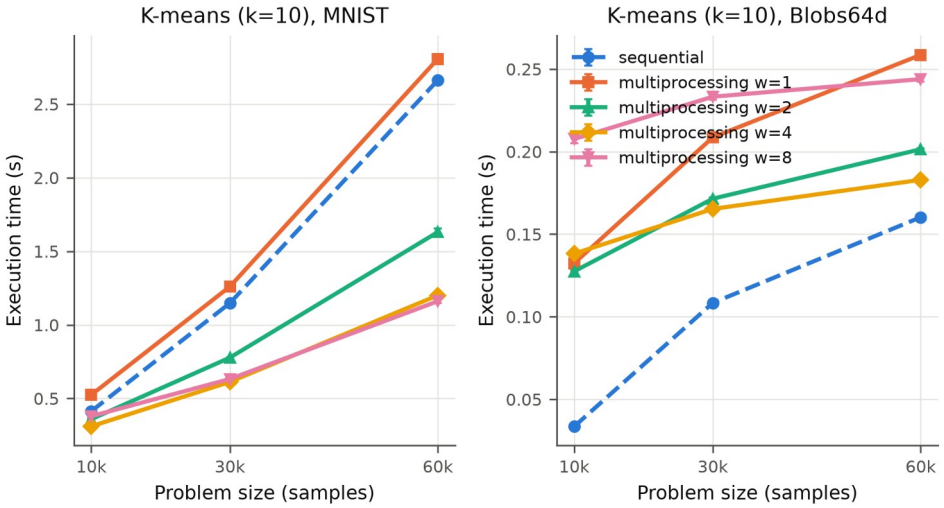


Figure B1. *S01 execution time vs number of samples (k = 10).*

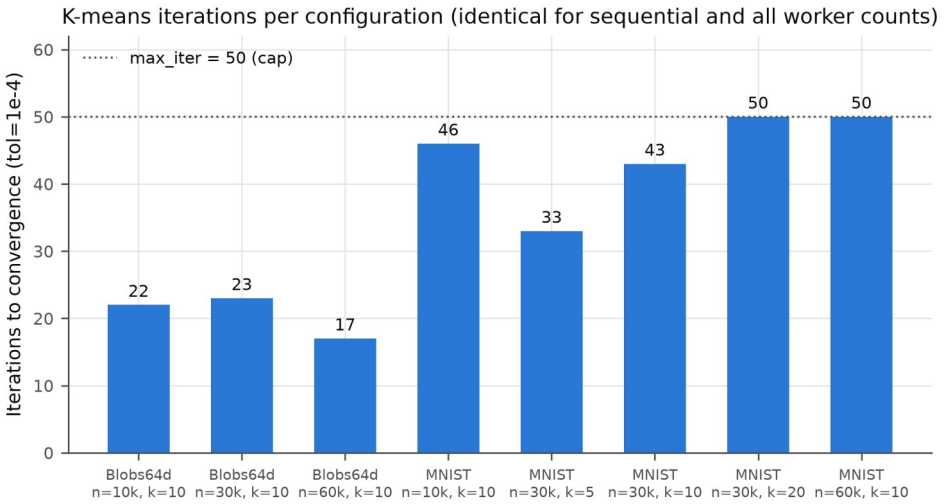


Figure B2. *S01 iterations to convergence per configuration (identical for all variants).*

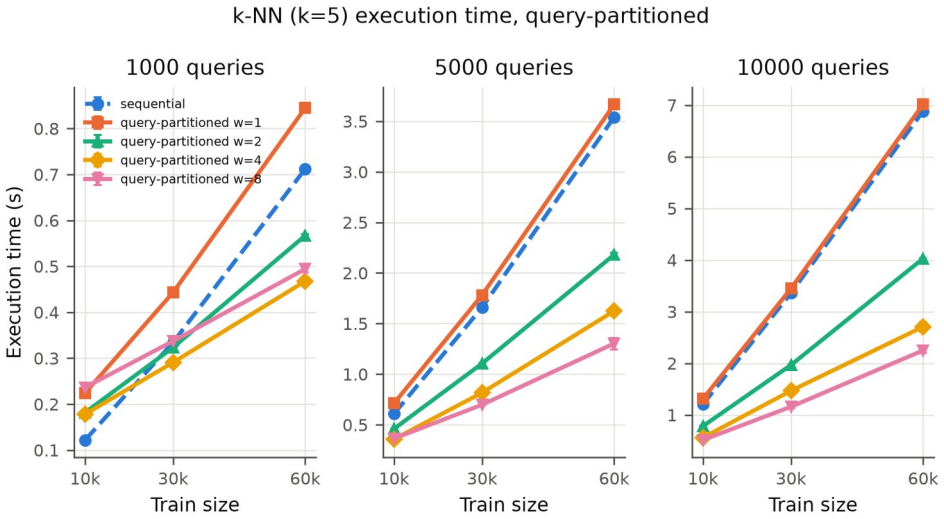


Figure B3. *S02 execution time vs training-set size for 1,000 / 5,000 / 10,000 queries (query-partitioned).*

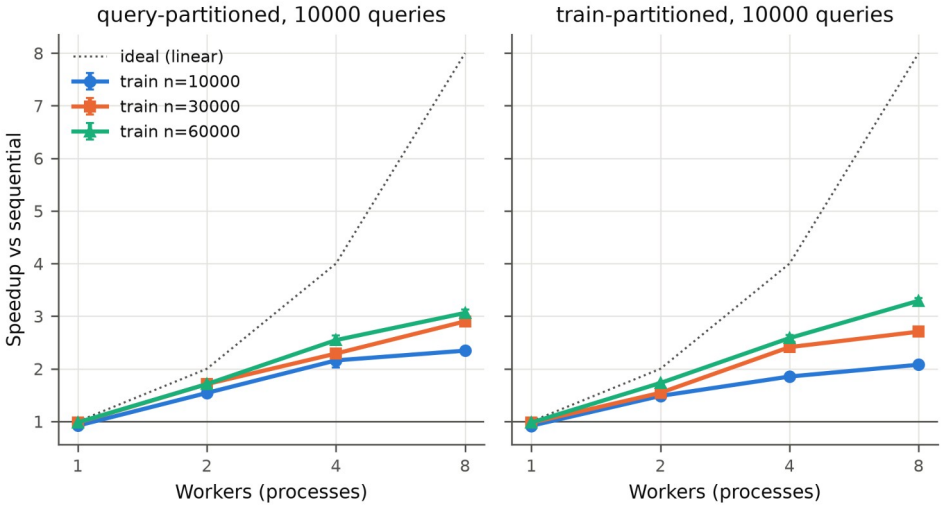


Figure B4. *S02 speedup vs workers per training-set size, 10,000 queries.*

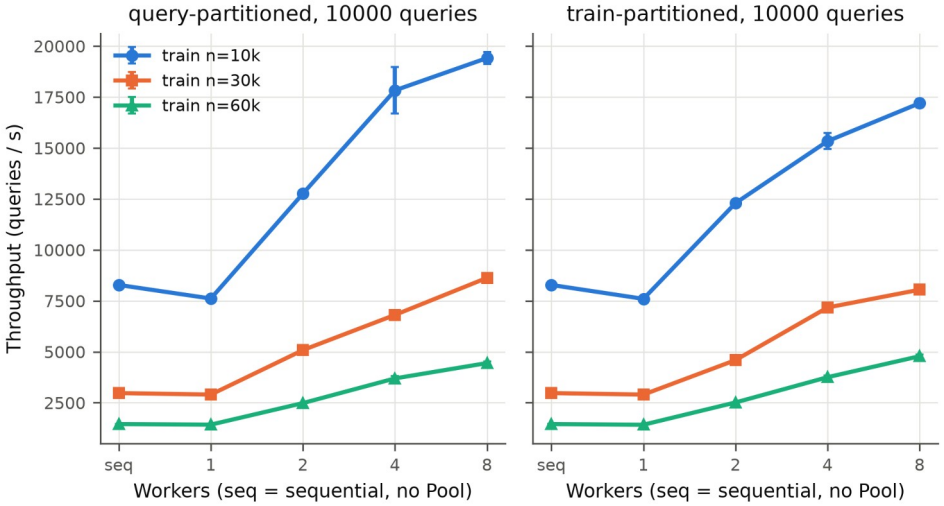


Figure B5. *S02 query throughput vs workers (10,000 queries) for both strategies; seq = pure sequential.*

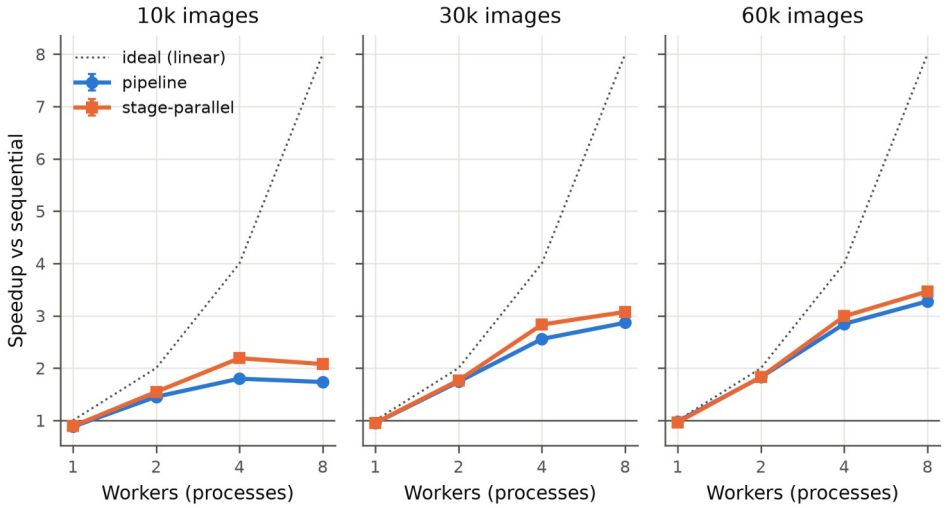


Figure B6. *S03 speedup vs workers per problem size for both variants.*

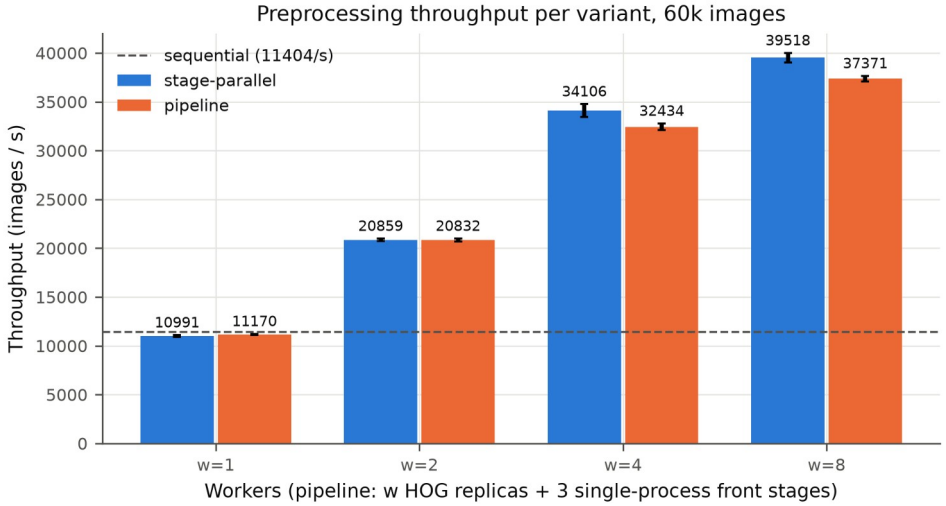


Figure B7. *S03 throughput at 60k images per variant and worker count; dashed line = pure sequential throughput.*

zsh - cs10: final benchmark sweep (excerpt of results/run_log.txt)

```
$ python run_all.py --mem-sampler track
sizes=[10000, 30000, 60000] workers=[1, 2, 4, 8] -> results/raw.csv
...
[S02] train=60000 queries=10000 k=5
  S02 q10000 n60000 -sequential-w1-r1    6.885s  mem=  398.4MB
  S02 q10000 n60000 -sequential-w1-r2    6.888s  mem=  398.4MB
  S02 q10000 n60000 -sequential-w1-r3    6.880s  mem=  398.4MB
  S02 q10000 n60000 -sequential-w1-r4    6.885s  mem=  398.4MB
  S02 q10000 n60000 -sequential-w1-r5    6.879s  mem=  398.4MB
  ...
  S02 q10000 n60000 /train-partitioned-multiprocessing-w8-r1  2.077s  mem= 1407.1MB
  S02 q10000 n60000 /train-partitioned-multiprocessing-w8-r2  2.086s  mem= 1406.2MB
  S02 q10000 n60000 /train-partitioned-multiprocessing-w8-r3  2.049s  mem= 1408.6MB
  S02 q10000 n60000 /train-partitioned-multiprocessing-w8-r4  2.099s  mem= 1407.8MB
  S02 q10000 n60000 /train-partitioned-multiprocessing-w8-r5  2.143s  mem= 1407.7MB
  ...
== s03 done in 356.9s
phase times (s): {'s01': 206.8, 's02': 784.8, 's03': 356.9} total: 1348.5
exit=0
```

Figure B8. *Console output of the final sweep (excerpt of results/run_log.txt; run identifiers abbreviated): the five timed sequential runs and the five train-partitioned w = 8 runs for 60k training images and 10,000 queries.*

zsh - cs10: correctness log and analysis

```
$ awk -F, 'NR>1{c[$9]++} END{for(k in c) print "Passed="k, c[k]}' results/correctness.csv
Passed=True 1854
$ wc -l results/raw.csv
841 results/raw.csv   (header + 840 timed runs)
$ python analysis/analyze.py
wrote derived.csv (840 rows), config_stats.csv (168), summary_table.csv/.md
$ python analysis/findings.py
wrote findings.md (17 anomalies flagged)
```

Figure B9. *Console output: correctness log (all 1,854 checks passed), raw-data size and the analysis scripts.*

zsh - cs10: memory-sampler diagnostic (results/sampler_diagnostic_log.txt)

```
$ python analysis/sampler_diagnostic.py --size 60000 --repeats 3
S01 K-means MNIST k=10       scan  rep1 2.952s
S01 K-means MNIST k=10       track rep1 2.669s
S01 K-means MNIST k=10       off   rep1 2.660s
S01 K-means MNIST k=10       scan  rep2 2.952s
S01 K-means MNIST k=10       track rep2 2.667s
S01 K-means MNIST k=10       off   rep2 2.658s
S01 K-means MNIST k=10       scan  rep3 2.956s
S01 K-means MNIST k=10       track rep3 2.662s
S01 K-means MNIST k=10       off   rep3 2.664s
S02 k-NN 1000 queries        scan  rep1 0.730s
S02 k-NN 1000 queries        track rep1 0.703s
S02 k-NN 1000 queries        off   rep1 0.702s
S02 k-NN 1000 queries        scan  rep2 0.725s
S02 k-NN 1000 queries        track rep2 0.716s
S02 k-NN 1000 queries        off   rep2 0.700s
S02 k-NN 1000 queries        scan  rep3 0.741s
S02 k-NN 1000 queries        track rep3 0.703s
S02 k-NN 1000 queries        off   rep3 0.701s
S03 preprocessing            scan  rep1 5.687s
S03 preprocessing            track rep1 5.174s
S03 preprocessing            off   rep1 5.231s
S03 preprocessing            scan  rep2 5.782s
S03 preprocessing            track rep2 5.226s
S03 preprocessing            off   rep2 5.208s
S03 preprocessing            scan  rep3 5.717s
S03 preprocessing            track rep3 5.188s
S03 preprocessing            off   rep3 5.187s
```

Figure B10. *Console output of the memory-sampler diagnostic (results/sampler_diagnostic_log.txt).*